# BUS 303 Signature Term Project: Competitive Analysis of the Starbucks and Dunkin' Donuts Beverage Menus

## Deliverable 1: Cleaning and Preparing the Starbucks Menu for Analysis

### Task 1.1: Import Pandas and Load the Starbucks CSV

In [1]:
```
import pandas as pd

sbux = pd.read_csv(
    'BUS303_Signature_Term_Project_Starbucks_Beverage_Menu.csv')

print('Loaded shape:', sbux.shape)
```

```
Loaded shape: (1205, 20)
```

### Task 1.2: Explore Rows with Blank Values

In [2]:
```
sbux.isnull().sum()
```

Out[2]:
```
Product_Name           1
Size                   1
Milk                   1
Whip                   1
Serving_Size_ml        1
Calories               1
Total_Fat_g            1
Saturated_Fat_g        2
Trans_Fat_g            1
Cholesterol_mg         1
Sodium_mg              1
Carbohydrates_g        1
Dietary_Fiber_g        1
Sugar_g                1
Protein_g              1
Vitamin_A_%DV          1
Vitamin_C_%DV          1
Calcium_%DV            1
Iron_%DV               1
Caffiene_milligrams    1
dtype: int64
```

### Task 1.3: View the Dataframe with the Head Function

In [3]:
```
sbux.head()
```

Out[3]:

| | Product_Name | Size | Milk | Whip | Serving_Size_ml | Calories | Total_Fat_g | Saturated_Fat_g | Trans_Fat_g | Cholesterol_mg | Sodium_mg | Carbohydrates_g | Dietary_Fiber_g | Sugar_g | Protein_g | Vitamin_A_%DV | Vitamin_C_%DV | Cal |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | Brewed Coffee - Dark Roast | Short | No Milk | No Whip | 236.0 | 3.0 | 0.1 | 0.0 | 0.0 | 0.0 | 5.0 | 0.0 | 0.0 | 0 | 0.3 | 0.0 | 0.0 | |
| **1** | Brewed Coffee - Dark Roast | Tall | No Milk | No Whip | 354.0 | 4.0 | 0.1 | 0.0 | 0.0 | 0.0 | 10.0 | 0.0 | 0.0 | zero | 0.5 | 0.0 | 0.0 | |
| **2** | Brewed Coffee - Dark Roast | Grande | No Milk | No Whip | 473.0 | 5.0 | 0.1 | 0.0 | 0.0 | 0.0 | 10.0 | 0.0 | 0.0 | 0 | 1.0 | 0.0 | 0.0 | |
| **3** | Brewed Coffee - Dark Roast | Venti® | No Milk | No Whip | 591.0 | 5.0 | 0.1 | 0.0 | 0.0 | 0.0 | 10.0 | 0.0 | 0.0 | 0 | 1.0 | 0.0 | 0.0 | |
| **4** | Brewed Coffee Traveler - Dark Roast | 1 - 236 mL serving | No Milk | No Whip | 236.0 | 3.0 | 0.1 | 0.0 | 0.0 | 0.0 | 5.0 | 0.0 | 0.0 | 0 | 0.3 | 0.0 | 0.0 | |

### Task 1.4: Remove Rows with All Blank Entries

In [4]:
```
sbux2 = sbux.dropna(how='all').copy()

print(
    'Shape after removing fully blank rows:',
    sbux2.shape
)
```

```
Shape after removing fully blank rows: (1204, 20)
```

### Task 1.5: Validate That the Blank Rows Were Removed

In [5]:
```
print(
    'Fully blank rows remaining:',
    sbux2.isnull().all(axis=1).sum()
)

sbux2.isnull().sum()
```

```
Fully blank rows remaining: 0
```

Out[5]:
```
Product_Name           0
Size                   0
Milk                   0
Whip                   0
Serving_Size_ml        0
Calories               0
Total_Fat_g            0
Saturated_Fat_g        1
Trans_Fat_g            0
Cholesterol_mg         0
Sodium_mg              0
Carbohydrates_g        0
Dietary_Fiber_g        0
Sugar_g                0
Protein_g              0
Vitamin_A_%DV          0
Vitamin_C_%DV          0
Calcium_%DV            0
Iron_%DV               0
```

# Deliverable Two.

## Cleaning and Preparing the Dunkin' Donuts Menu for Analysis.

**Task 2.1:**

Repeat the same steps as Task 1.1, except import the BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file.

In [1]:
```
# Task 2.1: Import pandas as pd to work with a package that can process dataframes.
# ENTER CODE BELOW THIS LINE.
import pandas as pd

dnkn = pd.read_csv(
    'BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv'
)

print('Loaded shape:', dnkn.shape)
dnkn.head()

# Import the BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file using the pandas package.
# Assign the file object name dnkn.
# ENTER CODE BELOW THIS LINE.
# Hint: dnkn =
```

```
Loaded shape: (780, 13)
```

Out[1]:

| | Category | Item | Calories | Total Fat (g) | Saturated Fat (g) | Trans Fat (g) | Cholesterol (mg) | Sodium (mg) | Carbs (g) | Fiber (g) | Sugars (g) | Protein (g) | Weight Watcher Pnts |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | Cold Brew Coffee | Cold Brew Coffee, Large | 5 | 0 | 0 | 0 | 0 | 20 | 0 | 0 | 0 | 1 | 4 |
| **1** | Cold Brew Coffee | Cold Brew Coffee, Medium | 5 | 0 | 0 | 0 | 0 | 15 | 0 | 0 | 0 | 0 | 5 |
| **2** | Cold Brew Coffee | Cold Brew Coffee, Small | 5 | 0 | 0 | 0 | 0 | 10 | 0 | 0 | 0 | 0 | 5 |
| **3** | Cold Brew Coffee | Cold Brew Coffee with Cream, Large | 120 | 12 | 6 | 0 | 35 | 65 | 2 | 0 | 2 | 2 | 126 |
| **4** | Cold Brew Coffee | Cold Brew Coffee with Cream, Medium | 90 | 9 | 4.5 | 0 | 25 | 45 | 1 | 0 | 1 | 2 | 93.5 |

**Task 2.2:**

- Examine the Dataset Structure
- Business analysts must first understand the structure of a dataset before making any modifications.

After reviewing the output:

- Which variables appear to be numerical?
- Which variables appear to be categorical?
- Are there any columns that appear to have an incorrect data type?

In [2]:
```
# Task 2.2: Examine the Dataset Structure

# ENTER CODE BELOW THIS LINE.
dnkn.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 780 entries, 0 to 779
Data columns (total 13 columns):
 #   Column               Non-Null Count  Dtype 
---  ------               --------------  ----- 
 0   Category             780 non-null    object
 1   Item                 780 non-null    object
 2   Calories             780 non-null    object
 3   Total Fat (g)        780 non-null    object
 4   Saturated Fat (g)    780 non-null    object
 5   Trans Fat (g)        780 non-null    object
 6   Cholesterol (mg)     780 non-null    object
 7   Sodium (mg)          780 non-null    object
 8   Carbs (g)            780 non-null    object
 9   Fiber (g)            780 non-null    object
 10  Sugars (g)           780 non-null    object
 11  Protein (g)          780 non-null    object
 12  Weight Watcher Pnts  780 non-null    object
dtypes: object(13)
memory usage: 79.3+ KB
```

The dataset contains 780 rows and 13 columns. Category and Item appear to be categorical variables. The remaining columns contain nutritional measurements and should be numerical. However, all 13 columns were imported as object data types. This text or special characters that prevent pandas from recognizing them as numerical.

**Task 2.3:**

- Explore Missing Values
- Missing values can negatively affect statistical analysis and visualizations. Before making any corrections, determine whether the dataset contains missing values.

Review the output and determine:

- Which columns contain missing values?
- How many missing values are present?

In [3]:
```
# Task 2.3: Explore Missing Values
```

The dataset contains 23 category values and 779 unique item names in 780 records. This supports the earlier finding that one item record is duplicated. The nutritional columns contain fewer unique values because many products share the same nutriti

**Task 2.7:**

- Examine the beverage categories.

Show the unique counts for all beverage categories. How would you interpret this output?

Are there any category labels that do not appear to be products? Are there any inconsistent category names? Are there unusual values?

In [7]:
```
# Task 2.7: Examine the Beverage Categories.

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn['Category'].
dnkn['Category'].value_counts()
```

Out[7]:
```
Category
Hot Coffee            88
Donuts                87
Hot Latte             63
Iced Coffee           63
Iced Latte            63
Sandwiches            55
Frozen Coffee         49
Hot Macchiato         45
Hot Cappuccino        43
Iced Macchiato        42
Iced Cappuccino       42
Iced Tea              33
Hot Chocolate         28
Muffins               18
Kolache               16
Cold Brew Coffee      11
Coolatta               9
Dunkin' Refreshers     9
Soft Serve             8
Hot Americano          3
Iced Americano         3
Hash Browns            1
Dunkin' Bowls          1
Name: count, dtype: int64
```

The Category column contains 23 different categories. Most categories describe beverages, but Donuts, Sandwiches, Muffins, Kolache, Hash Browns, and Dunkin' Bowls describe food. These food categories should be removed to create a beverage only otherwise.

**Task 2.8:**

- Explore Numerical Variables

The describe() function can be used to examine continuous variables and identify unusual values.

What are the:

- Minimum values
- Maximum values
- Means
- Missing values

In [8]:
```
# Task 2.8: Explore Numerical Variables

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn.describe
dnkn.describe(include='all')
```

Out[8]:

| | Category | Item | Calories | Total Fat (g) | Saturated Fat (g) | Trans Fat (g) | Cholesterol (mg) | Sodium (mg) | Carbs (g) | Fiber (g) | Sugars (g) | Protein (g) | Weight Watcher Pnts |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **count** | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 | 780 |
| **unique** | 23 | 779 | 87 | 50 | 29 | 5 | 37 | 112 | 120 | 9 | 122 | 32 | 362 |
| **top** | Hot Coffee | Ham and Cheese Rollups | 170 | 0 | 0 | 0 | 0 | 180 | 35 | 0 | 0 | 4 | 5 |
| **freq** | 88 | 2 | 38 | 242 | 269 | 755 | 222 | 24 | 30 | 508 | 33 | 94 | 13 |

The output displays counts, unique values, most common values, and frequencies instead of numerical statistics like means and minimums. This happens because the nutritional columns were imported as object data types. They have to be cleaned and summary statistics can be calculated.

**Task 2.9:**

- Identify Special Characters in the Total Fat (g) Column

Sometimes data contains special values, such as "zero", "N/A", "*", "<", "=", etc. A "special value" is any value that prevents a column from behaving as expected. For example, "zero" is not the same thing as 0. You need to identify them in the data and t numbers. There are various ways to do this. You can examine each column and look for these special characters.

Identify special characters in the Total Fat (g), Saturated Fat (g), Trans Fat (g), Cholesterol (mg), Sodium (mg), Fiber (g), Protein (g), and Carbs (g).

In [9]:
```
# Task 2.9a: Identify Special Characters in the Total Fat (g) Column

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn['Total Fat (g)'].
dnkn['Total Fat (g)'].unique()
```

Out[16]:
```
array(['0', '2', '1', '37', '27', '18', '112', '84', '56', '172', '129',
       '86', '113', '85', '31', '44', '57', '47', '63', '39', '55', '48',
       '36', '58', '33', '41', '34', '50', '40', '21', '42', '61', '43',
       '32', '52', '38', '59', '35', '45', '30', '49', '24', '176', '135',
       '95', '175', '134', '94', '180', '139', '97', '184', '142', '99',
       '141', '138', '183', '117', '88', '177', '160', '120', '80',
       '\xa0', '191', '147', '103', '195', '150', '105', '111', '83',
       '110', '13', '10', '7', '65', '64', '73', '70', '53', '71', '51',
       '87', '72', '66', '89', '82', '12', '9', '6', '26', '67', '5', '4',
       '46', '3', '28', '19', '17', '25', '23', '16', '15', '79', '78',
       '60', '11', '20', '54', '14', '69', '22', '75', '77', '29', '74',
       '68', '8', '92'], dtype=object)
```

In [17]:
```
# Task 2.9i: Identify Special Characters in the Sugars (g) Column

# ENTER CODE BELOW THIS LINE.
dnkn['Sugars (g)'].unique()
```

Out[17]:
```
array(['0', '2', '1', '36', '27', '18', '110', '83', '55', '167', '125',
       '107', '80', '53', '10', '22', '24', '21', '11', '34', '32', '26',
       '14', '33', '20', '13', '15', '19', '16', '28', '17', '23', '12',
       '25', '5', '29', '165', '126', '88', '162', '124', '87', '158',
       '121', '84', '172', '132', '92', '176', '135', '94', '175', '134',
       '169', '130', '91', '173', '133', '93', '108', '81', '54', '112',
       '56', '111', '159', '119', '151', '113', '75', '\xa0', '181',
       '139', '97', '185', '142', '99', '98', '109', '82', '9', '6', '47',
       '35', '46', '63', '31', '61', '30', '60', '45', '58', '43', '57',
       '38', '62', '50', '39', '70', '42', '71', '65', '69', '64', '51',
       '66', '40', '49', '37', '7', '3', '4', '59', '48', '74', '52',
       '68', '67', '78', '8', '44', '85'], dtype=object)
```

The nutritional columns contain the special nonbreaking space character '\xa0', which represents blank or missing information. The Cholesterol column also contains the value '<5'. These entries keep the nutritional columns from being treated as numer

**Alternate method to identify special values:**

Instead of identifying each column, we could use a loop to iterate over them and find special values. We are not doing it for the project. However, here is the code should you want to run it in a separate code block:

```
for col in dnkn.columns:
    if dnkn[col].dtype == 'object':
        print('\n', col)
        print(dnkn[col].unique()[:50])
```

**Task 2.10:**

- Create a Cleaned Dataframe

Create a new dataframe named dnkn1 that will contain all cleaning modifications.

In [18]:
```
# Task 2.10: Create a Cleaned Dataframe

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn1 = dnkn.
dnkn1 = dnkn.copy()

print('dnkn1 shape:', dnkn1.shape)
```

```
dnkn1 shape: (780, 13)
```

**Task 2.11:**

- Remove Food Items from the Category Rows

Examine the dataset and identify rows that appear to be food category labels rather than actual beverage products.

Remove these rows from the dataframe. Do this by creating a new list of food items and then remove them from the dataset.

In [19]:
```
# Task 2.11: Remove Food Items from the Category Rows

# ENTER CODE BELOW THIS LINE.

# Hint: food_items = ['Donuts', , , , , 'Kolache']

# Hint: dnkn2 = dnkn1[~dnkn1['Category'].isin(food_items)].
food_items = [
    'Donuts',
    'Sandwiches',
    'Muffins',
    'Kolache',
    'Hash Browns',
    'Dunkin' Bowls'
]

dnkn2 = dnkn1[
    ~dnkn1['Category'].isin(food_items)
].copy()

print('Rows removed:', len(dnkn1) - len(dnkn2))
print('Shape after removing food categories:', dnkn2.shape)

dnkn2['Category'].value_counts()
```

```
Rows removed: 178
Shape after removing food categories: (602, 13)
```

Out[19]:
```
Category
Hot Coffee            88
```

```
#       .str.strip())
for col in dnkn2.select_dtypes(
    include='object'
).columns:
    dnkn2[col] = (
        dnkn2[col]
        .str.replace('\xa0', '', regex=False)
        .str.strip()
    )
```

**Task 2.14:**

- Validate removal of all special characters.

Normally, you would validate all columns. I am providing some code for you to validate the removal of all special values. Run the code and inspect the output.

```
for col in dnkn.columns:
    if dnkn2[col].dtype == 'object':
        print('\n', col)
        print(dnkn2[col].unique()[:50])
```

In [23]:
```
# Task 2.14: Validate removal of all special characters.

# ENTER CODE BELOW THIS LINE.
# Hint: for col in dnkn2.columns:
#    if dnkn2[col].dtype == 'object':
#        print('\n', col)
#        print(dnkn2[col].unique()[:50])
for col in dnkn2.columns:
    if dnkn2[col].dtype == 'object':
        print('\n', col)
        print(dnkn2[col].unique()[:50])
```

```
 Category
['Cold Brew Coffee' 'Coolatta' 'Frozen Coffee' 'Hot Americano'
 'Hot Cappuccino' 'Hot Chocolate' 'Hot Coffee' 'Hot Latte' 'Hot Macchiato'
 'Iced Americano' 'Iced Cappuccino' 'Iced Coffee' 'Iced Latte'
 'Iced Macchiato' 'Iced Tea' 'Dunkin’ Refreshers' 'Soft Serve']

 Item
['Cold Brew Coffee, Large' 'Cold Brew Coffee, Medium'
 'Cold Brew Coffee, Small' 'Cold Brew Coffee with Cream, Large'
 'Cold Brew Coffee with Cream, Medium'
 'Cold Brew Coffee with Cream, Small'
 'Cold Brew Coffee with Cream and Sugar, Large'
 'Cold Brew Coffee with Cream and Sugar, Medium'
 'Cold Brew Coffee with Cream and Sugar, Small'
 'Nitro Cold Brew Coffee, Medium' 'Nitro Cold Brew Coffee, Small'
 'Blue Raspberry Coolatta, Large' 'Blue Raspberry Coolatta, Medium'
 'Blue Raspberry Coolatta, Small' 'Vanilla Bean Coolatta, Large'
 'Vanilla Bean Coolatta, Medium' 'Vanilla Bean Coolatta, Small'
 'Pineapple Coolatta, Large' 'Pineapple Coolatta, Medium'
 'Pineapple Coolatta, Small'
 'Caramel Swirl Frozen Coffee with Cream, Large'
 'Caramel Swirl Frozen Coffee with Cream, Medium'
 'Caramel Swirl Frozen Coffee with Cream, Small'
 'Caramel Swirl Frozen Coffee with Skim Milk, Large'
 'Caramel Swirl Frozen Coffee with Skim Milk, Medium'
 'Caramel Swirl Frozen Coffee with Skim Milk, Small'
 'Caramel Swirl Frozen Coffee with Whole Milk, Large'
 'Caramel Swirl Frozen Coffee with Whole Milk, Medium'
 'Caramel Swirl Frozen Coffee with Whole Milk, Small'
 'French Vanilla Swirl Frozen Coffee with Cream, Large'
 'French Vanilla Swirl Frozen Coffee with Cream, Medium'
 'French Vanilla Swirl Frozen Coffee with Cream, Small'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Large'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Medium'
 'French Vanilla Swirl Frozen Coffee with Skim Milk, Small'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Large'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Medium'
 'French Vanilla Swirl Frozen Coffee with Whole Milk, Small'
 'Frozen Coffee with Cream, Large' 'Frozen Coffee with Cream, Medium'
 'Frozen Coffee with Cream, Small' 'Frozen Coffee with Skim Milk, Large'
 'Frozen Coffee with Skim Milk, Medium'
 'Frozen Coffee with Skim Milk, Small'
 'Frozen Coffee with Whole Milk, Large'
 'Frozen Coffee with Whole Milk, Medium' 'Frozen Dunkaccino, Large'
 'Frozen Dunkaccino, Medium' 'Frozen Dunkaccino, Small'
 'Frozen Vanilla Chai/Vanilla Spice, Large']

 Calories
['5' '120' '90' '60' '260' '190' '130' '460' '350' '230' '790' '590' '390'
 '340' '1120' '860' '610' '830' '650' '470' '880' '680' '490' '1110' '640'
 '780' '500' '370' '250' '550' '410' '930' '710' '510' '400' '' '1160'
 '890' '630' '720' '480' '360' '240' '530' '10' '70' '45' '170' '110'
 '160']

 Total Fat (g)
['0' '12' '9' '6' '4.5' '3' '40' '31' '22' '5' '11' '8' '35' '26' '17'
 '13' '14' '10' '7' '' '4' '1' '0.5' '23' '21' '15' '16' '1.5' '2.5' '2'
 '18' '3.5' '20' '19' '27']

 Saturated Fat (g)
['0' '6' '4.5' '3' '3.5' '2.5' '1.5' '22' '17' '12' '7' '18' '14' '9' '15'
 '10' '5' '' '0.5' '4' '19' '8' '21' '13' '11' '1']
```

Out[25]:
```
Category              0
Item                  0
Calories              3
Total Fat (g)         3
Saturated Fat (g)     3
Trans Fat (g)         3
Cholesterol (mg)      3
Sodium (mg)           3
Carbs (g)             3
Fiber (g)             3
Sugars (g)            3
Protein (g)           3
Weight Watcher Pnts   3
dtype: int64
```

After the empty strings were converted to NaN values, three incomplete records became visible. Each nutritional column contains three missing values because the same three rows are missing their nutritional information.

**Task 2.17:**

- Create a new dnkn3 object and drop the NaN values from the dataframe.

Creation of the dnkn3 object is a good practice so the Python code does not get earlier dataframe versions mixed up.

In [26]:
```
# Task 2.17: Create a new dnkn3 object and drop the NaN values from the dataframe.

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn3 = dnkn2.dropna().
dnkn3 = dnkn2.dropna().copy()

print(
    'Shape after dropping rows with missing values:',
    dnkn3.shape
)
```

```
Shape after dropping rows with missing values: (599, 13)
```

**Task 2.18:**

- Validate removal of the NaN values from the dataframe.

In [27]:
```
# Task 2.18: Validate removal of the NaN values from the dataframe.

# ENTER CODE BELOW THIS LINE.
# Hint: dnkn3.isnull().
print(
    'Total missing values remaining:',
    dnkn3.isnull().sum().sum()
)

dnkn3.isnull().sum()
```

```
Total missing values remaining: 0
```

Out[27]:
```
Category              0
Item                  0
Calories              0
Total Fat (g)         0
Saturated Fat (g)     0
Trans Fat (g)         0
Cholesterol (mg)      0
Sodium (mg)           0
Carbs (g)             0
Fiber (g)             0
Sugars (g)            0
Protein (g)           0
Weight Watcher Pnts   0
dtype: int64
```

**Task 2.19:**

Convert Data Types

- Convert all numerical data types originally classified as objects.

Convert the variables to numeric data types. Create a new object called "numeric_columns" and create a list of all variables that are supposed to be classified as numeric. Then, us the following code to convert them:

```
for col in numeric_columns:
    dnkn3[col] = pd.to_numeric(dnkn3[col])
```

In [28]:
```
# Task 2.19: Convert Data Types

# ENTER CODE BELOW THIS LINE.
# Create an object called "numeric_columns" and list all columns.
# Hint: numeric_columns = [
#    'Calories',
#     'Total Fat (g)',
#     'Saturated Fat (g)',
#     'Trans Fat (g)',
#     'Cholesterol (mg)',
#     'Sodium (mg)',
#     'Carbs (g)',
#     'Fiber (g)',
#     'Sugars (g)',
#     'Protein (g)',
#     'Weight Watcher Pnts'
# ]
```

```
print("\n========== CATEGORY CHECK ==========")
print(dnkn3['Category'].value_counts())

print("\n========== SPECIAL VALUE CHECK ==========")
print("Cholesterol Values:")
print(dnkn3['Cholesterol (mg)'].unique())

print("\nTotal Fat Values:")
print(dnkn3['Total Fat (g)'].unique())

print("\n========== DATASET SHAPE ==========")
print(dnkn3.shape)

print("\n========== SUMMARY STATISTICS ==========")
print(dnkn3.describe().round(2))
```

```
========== MISSING VALUES ==========
Category               0
Item                   0
Calories               0
Total Fat (g)          0
Saturated Fat (g)      0
Trans Fat (g)          0
Cholesterol (mg)       0
Sodium (mg)            0
Carbs (g)              0
Fiber (g)              0
Sugars (g)             0
Protein (g)            0
Weight Watcher Pnts    0
dtype: int64

========== DATA TYPES ==========
Category               object
Item                   object
Calories                int64
Total Fat (g)         float64
Saturated Fat (g)     float64
Trans Fat (g)         float64
Cholesterol (mg)        int64
Sodium (mg)             int64
Carbs (g)               int64
Fiber (g)               int64
Sugars (g)              int64
Protein (g)             int64
Weight Watcher Pnts   float64
dtype: object

========== DUPLICATE RECORDS ==========
Duplicates: 0

========== CATEGORY CHECK ==========
Category
Hot Coffee            88
Iced Coffee           63
Hot Latte             63
Iced Latte            63
Frozen Coffee         47
Hot Macchiato         45
Iced Macchiato        42
Hot Cappuccino        42
Iced Cappuccino       42
Iced Tea              33
Hot Chocolate         28
Cold Brew Coffee      11
Coolatta               9
Dunkin' Refreshers     9
Soft Serve             8
Hot Americano          3
Iced Americano         3
Name: count, dtype: int64

========== SPECIAL VALUE CHECK ==========
Cholesterol Values:
[  0  35  25  20  15  10 130 100  70  40  30 105  80  55   5 125  50  45
  60]

Total Fat Values:
[ 0.  12.   9.   6.   4.5  3.  40.  31.  22.   5.  11.   8.  35.  26.
 17.  13.  14.  10.   7.   4.   1.   0.5 23.  21.  15.  16.   1.5  2.5
  2.  18.   3.5 20.  19.  27. ]

========== DATASET SHAPE ==========
(599, 13)

========== SUMMARY STATISTICS ==========
       Calories  Total Fat (g)  Saturated Fat (g)  Trans Fat (g)  \
count    599.00         599.00             599.00         599.00
mean     253.33           5.04               3.03           0.03
std      183.42           6.37               3.98           0.16
min        5.00           0.00               0.00           0.00
25%      130.00           0.00               0.00           0.00
50%      230.00           4.00               2.50           0.00
75%      340.00           8.00               4.50           0.00
max     1160.00          40.00              22.00           1.50

       Cholesterol (mg)  Sodium (mg)  Carbs (g)  Fiber (g)  Sugars (g)  \
```

In a two or three-paragraph layman's summary, tell your client what you did for Deliverable Two.

The original Dunkin' Donuts dataset contained 780 rows and 13 columns. Most of the columns contained nutritional measurements, but every column was initially identified as text. The initial missing-value check showed no missing data, but with furthe
characters that represented missing info. The dataset also contained one duplicated Ham and Cheese Rollups record, six food categories that did not belong in a beverage analysis, and a special "<5" entry in the Cholesterol column.

I created separate working copies of the dataset and removed the Donuts, Sandwiches, Muffins, Kolache, Hash Browns, and Dunkin' Bowls categories. This removed 178 food records, including the duplicate. I changed the "<5" cholesterol entry to 5, rer
converted the resulting empty strings to missing values, and removed three incomplete records. Next I converted all nutritional columns to numerical data types. The final cleaned dataset contains 599 rows and 13 columns with no missing values or dup
dnkn_clean.csv and reloaded to confirm that it was saved correctly.The cleaned dataset is now ready for reliable beverage comparisons, summary statistics, and further analysis.

In [ ]:

Liana Lai

BUS 303

Dr. Foy

Signature Term Project

# Reflection

Before taking this course, I expected Python to be more focused on memorizing code. I work in Excel regularly, to organize information and review data, and have limited experience coding in a business analysis context. Since this was a 7-week course, we covered a large amount of information in a short time. I learned that programming is not necessarily about memorizing commands, but understanding the problem, and breaking it into smaller, attainable steps.

I believe the course and Signature Term Project achieved the main learning objectives by giving me practical experience importing, examining, cleaning, validating, and exporting business data. The Starbucks and Dunkin' datasets looked organized at first, but each contained problems that could have affected the analysis. These problems included blank rows, missing values, incorrect data types, duplicate records, irrelevant food categories, misspelled column names, and special characters that hid missing information. Working through these issues showed me that data should never be assumed to be accurate or ready for analysis simply because it opens correctly in a spreadsheet or dataframe.

One of my takeaways is the importance of validating each cleaning decision instead of making several changes and assuming they worked. For example, in the Starbucks dataset, the word "zero" caused the Sugar column to be stored as text instead of numerical data. I had to replace that value, convert the column to a float, and then confirm that every value could be treated as a number. In the Dunkin' Donuts dataset, the first missing-value check reported zero missing values. Further inspection showed that some missing information was represented by nonbreaking space characters instead of standard NaN values. After removing the special characters and converting the empty strings to NaN, three incomplete records became visible. That was a good example of why an initial output does not always tell the whole story.

The problem-solving process was one of the most valuable parts of the project. I learned to create copies of dataframes before making major changes, inspect unique values, check data types, count missing values and duplicates, and verify the shape of the dataset after removing records. When something did not work, I had to look at the error message, check spelling and punctuation, and determine whether I had run the notebook cells in the correct order. Jupyter Notebook was helpful because it allowed me to see the code, explanation, and output together. At the same time, the order of the cells could be challenging because later code depended on earlier dataframes being created correctly.

Some parts of Python became easier once I understood the basic patterns. Commands such as head(), info(), isnull().sum(), dropna(), duplicated(), and to_csv() became easier to recognize and apply. Filtering records and creating lists of categories also became more understandable with practice. The more difficult parts were keeping track of multiple dataframe names, understanding why apparently numerical columns were classified as objects, and identifying invisible special characters. Syntax was also challenging because a missing quotation mark, parenthesis, bracket, or incorrect column name could prevent the code from running.

DataCamp helped introduce the commands and allowed me to practice them in smaller exercises. It was useful because the exercises provided immediate feedback and focused on one concept at a time. However, the Signature Term Project required more independent thinking. In DataCamp, the dataset and much of the structure were already provided. In the project, I had to follow a longer process, understand why each step was necessary, and make sure that changes made in one task did not create problems later. The combination of guided DataCamp exercises and the larger Jupyter project helped reinforce the material.

My perception of Python has changed because I now see how it can support business work instead of viewing it only as a programming language. In my work in construction estimating, I regularly use Excel and work with large amounts of information from drawings, estimates, bid forms, schedules, and vendor data. This information may come from different sources and may not always be organized consistently. Python could potentially help clean repeated datasets, standardize category names, identify missing information, compare lists, and automate tasks that would otherwise require manual Excel work. It could also create a repeatable record of the steps used to process information, which is valuable when someone else needs to understand or verify the workflow.

The hints were very important, especially because this was my first time completing a full project in Anaconda and Jupyter Notebook. They made the project less challenging, but I do not think that reduced the learning value. The hints helped identify the correct direction while I still had to enter the code, run it, interpret the results, and correct errors. Without the hints, I probably could have completed some parts through additional research, but it would have been difficult to complete the full project confidently within a seven-week course. As I gained more experience, I became better able to understand what the hints were asking me to accomplish instead of simply following them.

Before this course, I mainly viewed GitHub as a website for professional programmers. I now understand that it can also be used to store and share notebooks, code, documentation, and complete workflows. I still have much more to learn about version control and GitHub's full capabilities, but I can see it becoming useful for saving reusable scripts and showing how a data process was completed. In a professional setting, I would need to be careful not to upload confidential company or project information, but I could use it for personal practice projects, nonconfidential templates, and examples of my technical skills.

If the course had more time, I would have liked to learn more about connecting Python with Excel, automatically processing multiple files, creating dashboards, and producing more advanced visualizations. I would also be interested in learning how Python could compare two versions of a dataset and clearly identify what changed. That type of workflow could be useful in estimating when comparing drawing sets, bid information, or updated project data.

I plan to continue learning Python, although I expect to use it alongside Excel rather than as a complete replacement. My next step would be to practice with small, realistic tasks instead of trying to build a large program immediately. I could continue using DataCamp, review the notebooks from this course, and experiment with nonconfidential sample datasets related to estimating and business analysis. Overall, this course gave me a better understanding of how programming can make data cleaning more consistent, repeatable, and transparent. It also made Python feel more approachable and showed me how it could become a useful addition to my business and analytical skill set.

```
<class 'pandas.core.frame.DataFrame'>
Index: 1204 entries, 0 to 1204
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Product_Name         1204 non-null   object
 1   Size                 1204 non-null   object
 2   Milk                 1204 non-null   object
 3   Whip                 1204 non-null   object
 4   Serving_Size_ml      1204 non-null   float64
 5   Calories             1204 non-null   float64
 6   Total_Fat_g          1204 non-null   float64
 7   Saturated_Fat_g      1203 non-null   float64
 8   Trans_Fat_g          1204 non-null   float64
 9   Cholesterol_mg       1204 non-null   float64
 10  Sodium_mg            1204 non-null   float64
 11  Carbohydrates_g      1204 non-null   float64
 12  Dietary_Fiber_g      1204 non-null   float64
 13  Sugar_g              1204 non-null   float64
 14  Protein_g            1204 non-null   float64
 15  Vitamin_A_%DV        1204 non-null   float64
 16  Vitamin_C_%DV        1204 non-null   float64
 17  Calcium_%DV          1204 non-null   float64
 18  Iron_%DV             1204 non-null   float64
 19  Caffiene_milligrams  1204 non-null   float64
dtypes: float64(16), object(4)
memory usage: 197.5+ KB
```

## Task 1.11: Remove the Row Containing the NaN Value

In [11]:
```
df2 = sbux1.dropna().copy()

sbux3 = df2.copy()

print(
    'Shape after removing rows with missing values:',
    sbux3.shape
)
```

```
Shape after removing rows with missing values: (1203, 20)
```

## Task 1.12: Validate That the NaN Value Was Removed

In [12]:
```
sbux3.info()

print(
    'Missing values remaining:',
    sbux3.isnull().sum().sum()
)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1203 entries, 0 to 1204
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Product_Name         1203 non-null   object
 1   Size                 1203 non-null   object
 2   Milk                 1203 non-null   object
 3   Whip                 1203 non-null   object
 4   Serving_Size_ml      1203 non-null   float64
 5   Calories             1203 non-null   float64
 6   Total_Fat_g          1203 non-null   float64
 7   Saturated_Fat_g      1203 non-null   float64
 8   Trans_Fat_g          1203 non-null   float64
 9   Cholesterol_mg       1203 non-null   float64
 10  Sodium_mg            1203 non-null   float64
 11  Carbohydrates_g      1203 non-null   float64
 12  Dietary_Fiber_g      1203 non-null   float64
 13  Sugar_g              1203 non-null   float64
 14  Protein_g            1203 non-null   float64
 15  Vitamin_A_%DV        1203 non-null   float64
 16  Vitamin_C_%DV        1203 non-null   float64
 17  Calcium_%DV          1203 non-null   float64
 18  Iron_%DV             1203 non-null   float64
 19  Caffiene_milligrams  1203 non-null   float64
dtypes: float64(16), object(4)
memory usage: 197.4+ KB
Missing values remaining: 0
```

## Task 1.13: Correct the Caffeine Column Name

In [13]:
```
sbux3 = sbux3.rename(
    columns={
        'Caffiene_milligrams': 'Caffeine_milligrams'
    }
)
```

## Task 1.14: Validate the Corrected Column Name

In [14]:
```
sbux3.info()

print(
    'Caffeine_milligrams present:',
    'Caffeine_milligrams' in sbux3.columns
)

print(
    'Incorrect spelling remains:',
    'Caffiene_milligrams' in sbux3.columns
)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1203 entries, 0 to 1204
Data columns (total 20 columns):
```